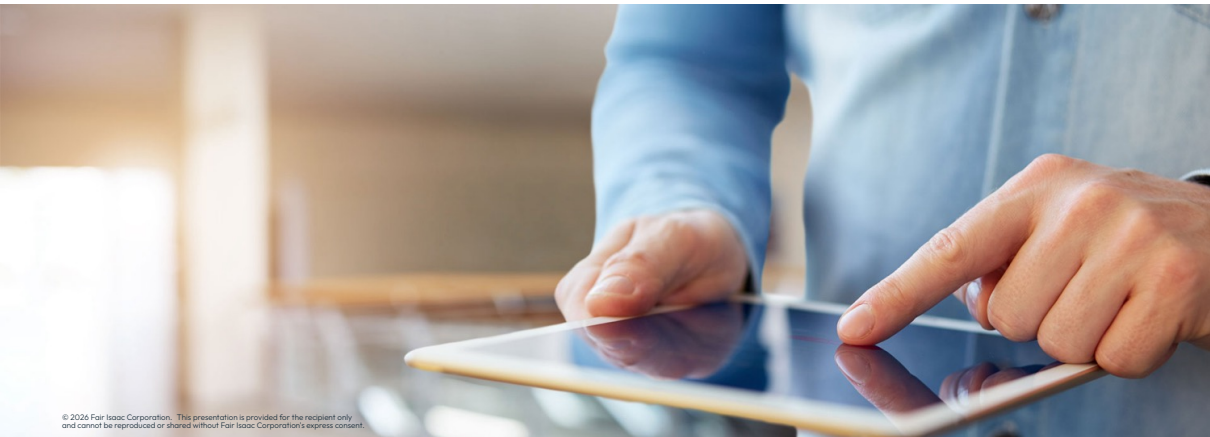






Modeling a basic optimization problem

Getting started and problem creation

- Importing the Xpress Python package:

- The `xpress` Python module can be imported as follows:

```
import xpress
```

- Since all types and methods must be called by prepending “`xpress.`”, it is advisable to alias the module name upon import:

```
import xpress as xp
```

- A complete list of methods and constants available in the module is obtained by running the Python command `dir(xpress)`

Getting started and problem creation

- Importing the Xpress Python package:

- The xpress Python module can be imported as follows:

```
import xpress
```

- Since all types and methods must be called by prepending “xpress.”, it is advisable to alias the module name upon import:

```
import xpress as xp
```

- A complete list of methods and constants available in the module is obtained by running the Python command `dir(xpress)`

- Problem creation:

- Create an empty optimization problem using `xpress.problem()`:

```
p = xp.problem()
```

- A name can be assigned to a problem upon creation using the `name` argument:

```
p = xp.problem(name="My first problem")
```

Create and add decision variables

- Use the `problem.addVariable()` function to create decision variables and directly add them to the optimization problem:

```
p.addVariable(name, lb, ub, threshold, vartype)
```

- All **parameters are optional**:
 - `name`: string containing the name of the variable. A default name is assigned if not specified
 - `lb, ub`: lower bound (0 by default) and upper bound (+inf by default), respectively
 - `threshold`: must be defined for semi-continuous, semi-integer, and partially integer variables, with a value between their lower and upper bounds
 - `vartype`: the variable type, one of the six following types:
 - `xp.continuous` for continuous variables
 - `xp.binary` for binary variables (`lb, ub`: are further restricted to 0 and 1, respectively)
 - `xp.integer` for integer variables
 - `xp.semicontinuous` for semi-continuous variables
 - `xp.semiinteger` for semi-integer variables
 - `xp.partiallyinteger` for partially integer variables

Create and add decision variables

- Variables added to an Xpress problem are constrained to be **nonnegative by default**:
 - To add a free variable, one must specify its lower bound as `-xp.infinity`:

```
x = p.addVariable(lb=-xp.infinity)
```

Create and add decision variables

- Variables added to an Xpress problem are constrained to be **nonnegative by default**:
 - To add a free variable, one must specify its lower bound as `-xp.infinity`:

```
x = p.addVariable(lb=-xp.infinity)
```

- A **set of variables** can be created at once by using **lists or dictionaries**:

```
# with lists
```

```
L = range(20)
```

```
x = [p.addVariable(ub=1) for i in L]
```

```
y = [p.addVariable(vartype=xp.binary) for i in L]
```

```
# with dictionaries
```

```
LC = ['Seattle', 'Miami', 'Omaha', 'Charleston']
```

```
z = {i: p.addVariable(vartype=xp.integer) for i in LC}
```



Hint: *Dictionaries allow us to refer to such variables using the names in LC, for instance `z['Seattle']`, `z['Charleston']`.*

Create and add decision variables

- **Variable names** can be useful when saving a problem to a file and when querying the problem for the value of a variable in an optimal solution:
 - When querying for a variable or expression containing that variable, its name will be printed rather than the Python object used in programming:
 - This allows for querying a problem using both the variable object and its name

Create and add decision variables

- **Variable names** can be useful when saving a problem to a file and when querying the problem for the value of a variable in an optimal solution:
 - When querying for a variable or expression containing that variable, its name will be printed rather than the Python object used in programming:
 - This allows for querying a problem using both the variable object and its name
 - If a variable is not specified with a name by the user, it will be assigned a “C” followed by a sequence number:

```
v = p.addVariable(lb=-1, ub=2)
print(v)
>>> C1
```

- If a variable name is explicitly specified:

```
x = p.addVariable(name='myvar')
print(v + 2 * x)
>>> C1 + 2 myvar
```

Create and add decision variables

- Use the function `problem.addVariables()` for creating an indexed set of variables:

```
p.addVariables(*indices, name, lb, ub, threshold, vartype)
```

- Parameter `*indices` stands for one or more arguments, each a list, a set, or a positive integer:
 - Produces as many variables as can be indexed with all combinations from the lists/sets
- If `*indices` consists of one list/set, a variable will be created for each element in the list:

```
myvar = p.addVariables(['a','b','c'], lb=-1, ub=+1)
```

- Yields `myvar['a']`, `myvar['b']`, and `myvar['c']`

Create and add decision variables

- Use the function `problem.addVariables()` for creating an **indexed set of variables**:

```
p.addVariables(*indices, name, lb, ub, threshold, vartype)
```

- Parameter `*indices` stands for one or more arguments, each a list, a set, or a positive integer:
 - Produces as many variables as can be indexed with all combinations from the lists/sets
- If `*indices` consists of **one list/set**, a variable will be created for each element in the list:

```
myvar = p.addVariables(['a','b','c'], lb=-1, ub=+1)
```

- Yields `myvar['a']`, `myvar['b']`, and `myvar['c']`
- In case of **more than one list/set**, the Cartesian product of these lists/sets provides the indexing space of the result in the form of a **dictionary indexed by tuples**:

```
y = p.addVariables(['a','b','c','d'], [100, 120, 150], vartype=xp.integer)
```

- Results in 12 variables `y['a',100]`, `y['a',120]`, `y['a',150]`, ..., `y['d',150]`

Create and add constraints

- Constraints can be created in a natural way by using the overloaded operators `<=`, `==`, `>=`:

```
myconstr = x1 + x2 * (x2 + 1) <= 4  
myconstr2 = xp.exp(xp.sin(x1)) + x2 * (x2**5 + 1) <= 4
```

- Use the `problem.addConstraint()` method to add constraints to a problem:

```
p.addConstraint(c1, c2, ...)
```

- Where `c1, c2, ...` are constraints or list/tuples/array of constraints
- Can be added directly, for example:

```
p.addConstraint(v1 + xp.tan(v2) <= 3)
```

Create and add constraints

- Constraints can be created in a natural way by using the overloaded operators `<=`, `==`, `>=`:

```
myconstr = x1 + x2 * (x2 + 1) <= 4  
myconstr2 = xp.exp(xp.sin(x1)) + x2 * (x2**5 + 1) <= 4
```

- Use the `problem.addConstraint()` method to add constraints to a problem:

```
p.addConstraint(c1, c2, ...)
```

- Where `c1, c2, ...` are constraints or list/tuples/array of constraints
- Can be added directly, for example:

```
p.addConstraint(v1 + xp.tan(v2) <= 3)
```

- Several constraints (or lists of constraints) can be added at once:

```
p.addConstraint(myconstr, myconstr2)  
p.addConstraint(x[i] + y[i] <= 2 for i in range(10))
```

Naming constraints with `addConstraint()`

- Since Xpress 9.9, `problem.addConstraint()` accepts an optional `name=` argument to attach a human-readable label to each constraint at creation time:

```
p.addConstraint(x + y <= 100, name="budget")
```

- Named constraints make infeasibility analysis and log output much easier to interpret

Naming constraints with `addConstraint()`

- Since Xpress 9.9, `problem.addConstraint()` accepts an optional `name=` argument to attach a human-readable label to each constraint at creation time:

```
p.addConstraint(x + y <= 100, name="budget")
```

- Named constraints make infeasibility analysis and log output much easier to interpret
- The `name=` argument supports three patterns:
 - **Single constraint:** pass a string directly

```
p.addConstraint(x + y <= 100, name="budget")
```

- **Loop with f-string:** each constraint gets a unique name

```
for t in PERIODS:  
    p.addConstraint(qty[t] >= demand[t], name=f"min_demand_{t}")
```

- **Generator with name prefix:** Xpress appends the index automatically, e.g. `capacity(0)`, `capacity(1)`, *etc.*

```
p.addConstraint((qty[t] <= 50 for t in PERIODS), name="capacity")
```

Create and add constraints

- Lists and dictionaries can also be used to create constraints:

```
LC = ['Seattle', 'Miami', 'Omaha', 'Charleston']  
constr = [x[i] <= y[i] for i in LC]  
cliq = {(i,j): x[i] + x[j] <= 1 for i in LC for j in L if i != j}  
p.addConstraint(constr, cliq)
```



Hint: *By using dictionaries, each constraint can be referred to with pairs of names, e.g. cliq['Seattle', 'Miami'].*

Create and add constraints

- Lists and dictionaries can also be used to create constraints:

```
LC = ['Seattle', 'Miami', 'Omaha', 'Charleston']  
constr = [x[i] <= y[i] for i in LC]  
cliq = {(i,j): x[i] + x[j] <= 1 for i in LC for j in L if i != j}  
p.addConstraint(constr, cliq)
```



Hint: *By using dictionaries, each constraint can be referred to with pairs of names, e.g. cliq['Seattle', 'Miami'].*

- For compactness, formulate constraints with the `xpress.Sum()` operator to define sums of variables or expressions:

```
p.addConstraint(xp.Sum(x) <= 1)  
p.addConstraint(xp.Sum([y[i] for i in range(10)]) <= 1)  
p.addConstraint(xp.Sum([x[i]**5 for i in range(9)]) <= x[9])
```

Create and add constraints

- Alternatively, use the method `xpress.constraint()` to be able to provide a name for the constraint:

```
xp.constraint(constraint, name)  
xp.constraint(body, type, rhs, lb, ub, name)
```

- Can be passed a `constraint` object directly or defined via its members `body`, `type`, `rhs`
- For the second case, `type` of constraint can be `xp.leq`, `xp.geq`, `xp.eq`, or `xp.rng`

Create and add constraints

- Alternatively, use the method `xpress.constraint()` to be able to provide a name for the constraint:

```
xp.constraint(constraint, name)
xp.constraint(body, type, rhs, lb, ub, name)
```

- Can be passed a constraint object directly or defined via its members `body`, `type`, `rhs`
- For the second case, type of constraint can be `xp.leq`, `xp.geq`, `xp.eq`, or `xp.rng`
- Examples of use:
 - Passing a constraint expression directly as an argument and defining a name:
`c1 = xp.constraint(x1 + 2*x2 <= 3, name="myconstraint1")`
 - Passing the `body`, `type` and `rhs` arguments instead of the constraint object:
`c2 = xp.constraint(body=x1 + 2*x2, type=xp.leq, rhs=3, name="myconstraint2")`
 - Can be particularly useful to define range constraints by passing the type as `xp.rng` and `lb`, `ub`:
`c3 = xp.constraint(body=x1 + 2*x2, type=xp.rng, lb=0, ub=3, name="myconstraint3")`
 - This will add the range constraint $0 \leq x1 + 2x2 \leq 3$

Create and add the objective function

- The method `problem.setObjective()` sets the objective function of a problem:

```
p.setObjective(objective, sense=xp.ObjSense.MINIMIZE)
```

- Where `objective` is a required expression defining the objective, and the optional argument `sense` can be either `xp.ObjSense.MINIMIZE` or `xp.ObjSense.MAXIMIZE`

Create and add the objective function

- The method `problem.setObjective()` sets the objective function of a problem:

```
p.setObjective(objective, sense=xp.ObjSense.MINIMIZE)
```

- Where `objective` is a required expression defining the objective, and the optional argument `sense` can be either `xp.ObjSense.MINIMIZE` or `xp.ObjSense.MAXIMIZE`

- By default, the objective function is to be minimized:

```
p.setObjective(xp.Sum([y[i]**2 for i in range (10)]))
```

- Define `sense=xp.ObjSense.MAXIMIZE` to change the optimization sense to maximization:

```
obj = v1 + 3 * v2  
p.setObjective(obj, sense=xp.ObjSense.MAXIMIZE)
```



Thank You

www.fico.com/optimization